Deep Reinforcement Learning

Module 5: Multi-Agent Reinforcement Learning

From single-agent RL to coordinated decision-making

Mustafa Özger

Aalborg University

31 March - Spring 2026

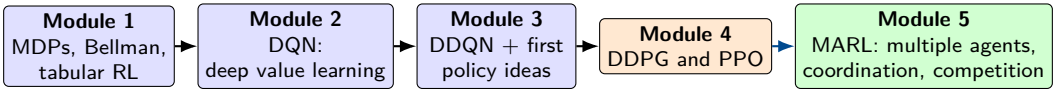
Why this module matters

- Modules 1–4 mostly assumed one learner or one centralized policy.
- Many real systems involve multiple decision-makers learning at the same time: robot teams, traffic intersections, wireless nodes, markets, and games.
- Once other agents learn too, the environment becomes non-stationary from any one agent's perspective.
- Rewards, observations, and control may be decentralized even when the team objective is shared.

Teaching goal

Students should leave this lecture understanding how MARL changes the problem formulation, why centralized training with decentralized execution (CTDE) is so common, and how the main algorithm families fit together.

Where Module 5 fits in the course



- Module 5 extends actor–critic and value-learning ideas to settings with multiple agents.
- The main new ingredients are Markov games, decentralized information, coordination/competition, and learning under non-stationarity.
- Big picture: single-agent RL learns against a fixed environment; MARL learns in an ecosystem of other learners.

Learning outcomes

After this lecture, students should be able to:

- define a stochastic / Markov game and explain how it extends an MDP,
- distinguish cooperative, competitive, and mixed multi-agent settings,
- explain the core MARL challenges: non-stationarity, credit assignment, partial observability, and scaling of joint action spaces,
- explain centralized training with decentralized execution (CTDE),
- compare independent learners, value-factorization methods, centralized-critic actor–critic methods, and self-play,
- read the key ideas behind VDN, QMIX, MADDPG, COMA, and MAPPO at a conceptual level,
- evaluate MARL experiments using the right partner/opponent protocols, seeds, and metrics.

Roadmap

1. What changes in multi-agent RL?
2. Markov games and the core difficulties
3. CTDE and the main algorithm families
4. Coordination, competition, and communication
5. Practical training, evaluation, and applications

How to follow

If you feel lost, ask three questions:

1. What is the game structure?
2. What information is centralized during training?
3. What does each agent actually know and do at execution time?

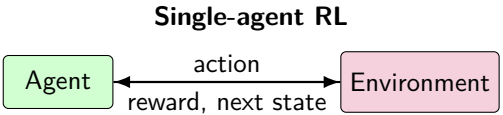
Notation used in this lecture

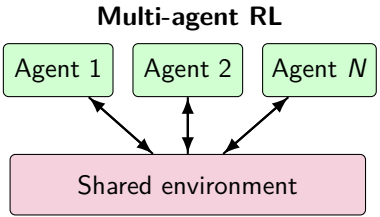
N	number of agents
s_t	global state (if available during training)
o_t^i	local observation of agent i
a_t^i	action of agent i
$\boldsymbol{a}_t = (a_t^1, \ldots, a_t^N)$	joint action
r_t^i	reward of agent i; sometimes all agents share a team reward r_t
$\pi_i(a^i \mid o^i)$	decentralized policy of agent i
Q_{tot}	centralized team action-value used by value-factorization methods

Important convention

At execution time, each decentralized policy should depend only on information realistically available to that agent. Extra global state is a training-time privilege, not an execution-time assumption.

From one learner to many learners





- The key structural change is that each agent must learn while other agents also adapt.
- Other policies are no longer just "part of the environment"; they are moving targets.

Three common multi-agent settings

Cooperative

- Shared team objective
- Examples: robot teams, traffic control, warehouse coordination
- Need coordination and credit assignment

Competitive

- Agents have opposing rewards
- Examples: pursuit–evasion, zero-sum games
- Self-play and opponent adaptation matter

Mixed / general-sum

- Some incentives align; others conflict
- Examples: markets, negotiation, autonomous driving interaction
- Need both coordination and strategic reasoning

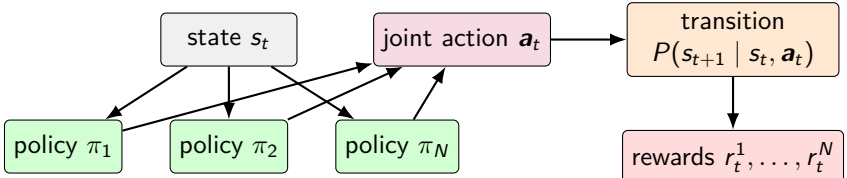
Important reminder

MARL is not only "many cooperative agents." The reward structure determines what kind of game you are solving, which in turn determines whether self-play, equilibrium thinking, or team coordination is the central issue.

Stochastic / Markov games: the formal model

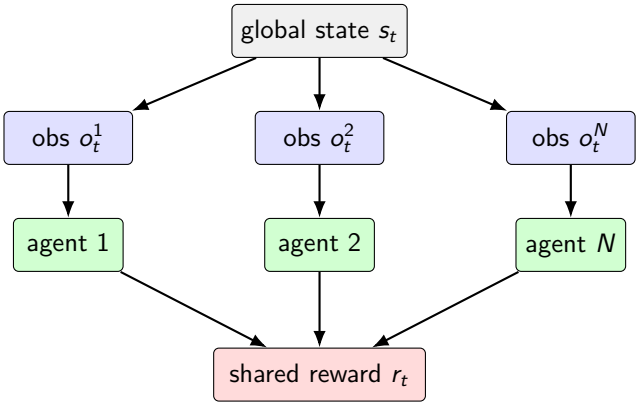
- A stochastic game extends an MDP to multiple agents.
- At each step, all agents choose actions, the environment transitions, and each agent receives its own reward.
- An MDP is the special case with one agent.

$$(\mathcal{S}, \mathcal{A}_1, \ldots, \mathcal{A}_N, P, r_1, \ldots, r_N, \gamma)$$



Cooperative partial observability: the Dec-POMDP view

- In many cooperative problems, agents do not observe the full state; each agent sees only a local observation.
- The team often has a shared reward, but execution must be decentralized.
- This is why local observations, memory, communication, and CTDE become so important.



decentralized execution
each policy acts from its own local history / memory

Part I: Why MARL is hard

Core difficulties

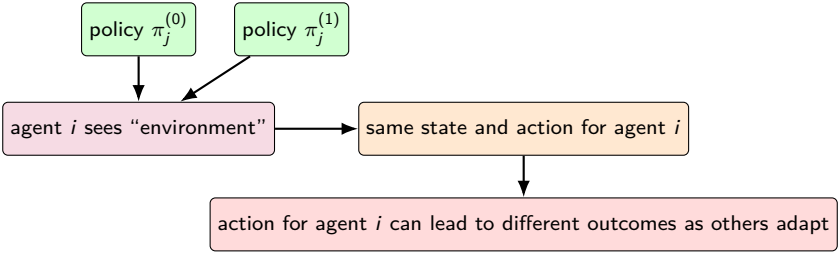
- **Non-stationarity:** while one agent learns, the other agents also change their policies.
- **Partial observability:** each agent usually sees only local information rather than the full global state.
- **Credit assignment:** team rewards do not clearly identify which agent caused success or failure.
- **Strategic interaction:** rewards may align, conflict, or mix both, depending on the game.

One-sentence idea

MARL is harder than single-agent RL because each learner must adapt inside a partially observed game whose dynamics change as the other agents learn.

Challenge 1: non-stationarity from co-learning agents

- If agent j changes its policy, then the transition and reward distribution seen by agent i also change.
- From the point of view of any one learner, the environment is no longer stationary.
- This breaks assumptions behind naive Q-learning, replay, and convergence intuition from single-agent RL.



Key consequence

Experience generated against old teammate/opponent policies can become stale much faster than in single-agent RL.

Challenge 2: multi-agent credit assignment

- A team reward says whether the outcome was good, but not which agent actions made it good.
- This creates lazy-agent and spurious-reward problems: one agent may do all the work, or individual updates may chase misleading signals.
- Good MARL algorithms try to produce a more informative per-agent learning signal without breaking decentralized execution.

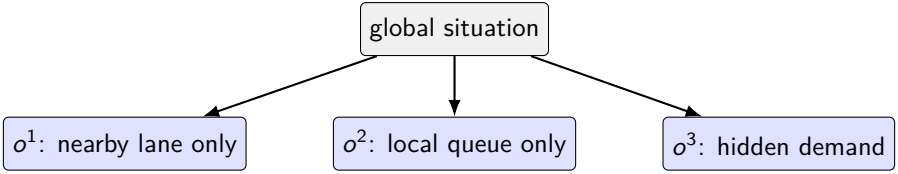
joint reward $= +1$ | agent 1 moved well | agent 2 hesitated | agent 3 communicated

Which agent should get the strongest update?

Typical solution idea: use counterfactual baselines, centralized critics, or value-factorization structures so the gradient or TD target better reflects each agent's contribution.

Challenge 3: partial observability and decentralized information

- Each agent may know only a local slice of the world: nearby sensors, private goals, or partial history.
- Coordination requires reasoning about what others know, what they are likely to do, and when communication matters.
- Memory, recurrence, belief tracking, and learned communication are often more important in MARL than in simple fully observed benchmarks.



Execution-time constraint

Even if a centralized trainer can see the whole state, each deployed agent must still act from whatever sensors, messages, and history it really has.

Challenge 4: joint-action explosion

- If each of N agents has $|\mathcal{A}|$ actions, the joint action space grows as $|\mathcal{A}|^N$.
- Learning a fully centralized $Q(s, a^1, \ldots, a^N)$ quickly becomes expensive even when each individual action set is small.

$$|\mathcal{A}_{\text{joint}}| = \prod_i |\mathcal{A}_i|$$

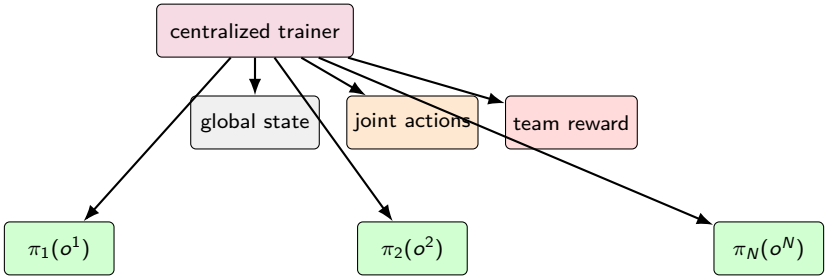
Agents	Joint actions if each agent has 5 actions
1	5
2	25
4	625
8	390,625

Two practical responses

- **Value factorization:** preserve decentralized greedy action selection while training a centralized team objective.
- **Centralized critics:** avoid joint argmax directly by using actor–critic style updates.

Centralized training, decentralized execution (CTDE)

- CTDE is the dominant design pattern in modern MARL.
- Training may use global state, all agent actions, or richer simulator information.
- Execution still uses decentralized policies that depend only on local observations (and perhaps messages/history).



deployed agents use only local information

Why CTDE works well

Extra centralized information can reduce variance, improve credit assignment, and stabilize learning, while still respecting decentralized deployment constraints.

Baseline idea: independent learners

Setup

- Agent 1 runs DQN / PPO / actor–critic
- Agent 2 runs its own learner
- All other agents become part of the "environment"

Pros / cons

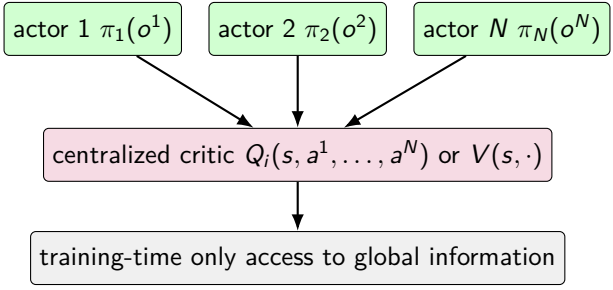
- **Pros:** minimal code changes from single-agent RL, scales easily, useful baseline.
- **Cons:** severe non-stationarity, poor credit assignment, little explicit coordination mechanism.
- In cooperative settings, independent learners often work only when the task is simple or the environment is forgiving.

Rule of thumb

Independent learners are a baseline, not a full solution. They tell you how much benefit comes from explicit multi-agent structure.

Centralized critics: keep decentralized actors, enrich the critic

- A centralized critic can condition on extra information such as global state, teammate actions, or opponent actions.
- The actors remain decentralized: each actor still maps local information to its own action.
- This often lowers gradient variance and gives a cleaner learning signal.

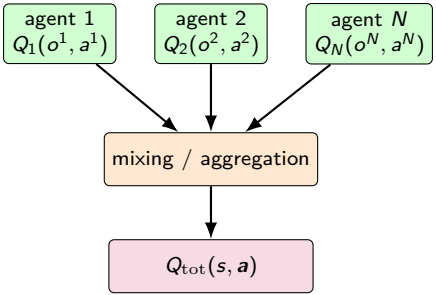


Policy-gradient view

Increase $\log \pi_i(a_i \mid o_i)$ when the advantage $\hat{A}_i$ is positive.

Value factorization: keep a team objective, execute locally

- In fully cooperative settings, we often want to optimize a team action-value while still allowing each agent to act from local information.
- Value-factorization methods learn per-agent utilities and combine them into a centralized team value during training.



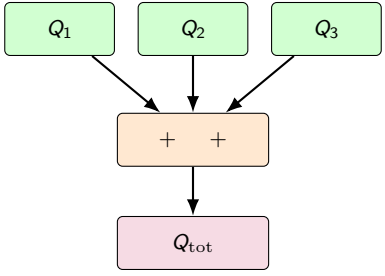
Execution-time benefit

If the structure is chosen carefully, each agent can still pick its own greedy action locally even though training optimized a centralized team value.

VDN: Value-Decomposition Networks

- VDN uses the simplest possible factorization: the team value is the sum of per-agent action-values.
- This is easy to optimize and preserves decentralized greedy action selection.
- Its limitation is expressiveness: some coordination patterns cannot be represented well by a purely additive team value.

$$Q_{\text{tot}}(\boldsymbol{s}, \boldsymbol{a}) = \sum_i Q_i(o^i, a^i)$$

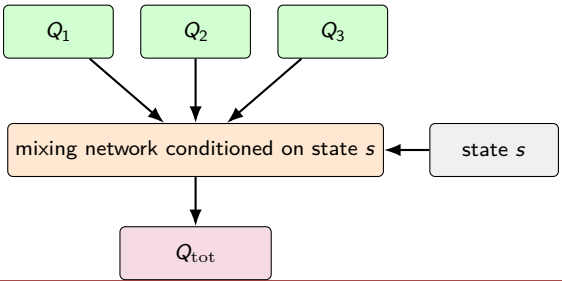


When to remember VDN

Think: cooperative tasks, shared reward, and a baseline factorization that is simple and interpretable.

QMIX: richer mixing under a monotonicity constraint

- QMIX replaces the simple sum in VDN with a learned mixing network conditioned on global state.
- It imposes a monotonicity constraint so that improving an individual utility cannot decrease the team value.
- This keeps decentralized greedy action selection tractable while allowing much richer team-value structure than VDN.



Intuition

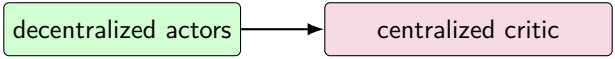
QMIX says: use centralized information to learn a better team value, but preserve enough structure that each agent can still act greedily from its own utility.

COMA: centralized critic with a counterfactual baseline

- COMA is a cooperative actor–critic method designed to improve credit assignment.
- A centralized critic estimates the effect of joint actions, while each decentralized actor gets a counterfactual advantage.
- The baseline asks: how good would this state have been if agent i had acted differently while others stayed fixed?

Counterfactual idea

counterfactual advantage = joint value − expected value if only agent i changed action

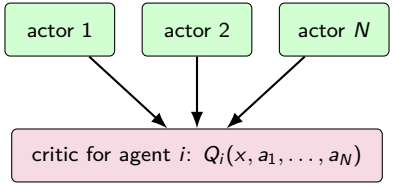


Why it matters

COMA is one of the clearest examples of using a centralized critic specifically to solve team credit assignment, not just to reduce variance.

MADDPG: multi-agent actor–critic for mixed settings

- MADDPG extends deterministic actor–critic ideas to multiple agents.
- Each agent has its own actor, but the critic for agent i can condition on global training information and other agents' actions.
- This is especially useful in mixed cooperative-competitive environments and for continuous actions.



Update intuition

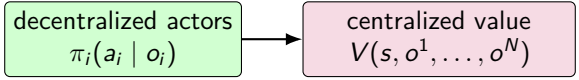
Update actor i toward actions that increase the critic value $Q_i(x, a_1, \ldots, a_N)$.

Memory hook

Think of MADDPG as "DDPG + CTDE + one critic per agent that knows more at training time than the actor will know at execution time."

MAPPO: PPO adapted to the cooperative multi-agent setting

- MAPPO keeps the PPO-style on-policy actor update but uses centralized information in the value function during training.
- Actors stay decentralized at execution time; the main stability comes from PPO clipping and careful implementation details.
- Empirically, PPO-style methods turned out to be stronger MARL baselines than many people expected.



Practical message

When implemented carefully, MAPPO is often a very strong cooperative MARL baseline and a good starting point before more specialized methods.

Algorithm families at a glance

Family	Data regime	Policy type	Best remembered for	Typical methods
Independent learners	off- or on-policy	local	baseline simplicity	independent DQN / PPO
Value factorization	off-policy	local greedy	cooperative team values	VDN, QMIX
Centralized critic	off- or on-policy	decentralized actors	better learning signal	COMA, MADDPG, MAPPO
Self-play / league	varies	depends on base algorithm	competitive adaptation	self-play with PPO, population methods

Interpretation

These are not mutually exclusive worldviews. CTDE can be combined with on-policy or off-policy learning, and self-play can sit on top of an actor–critic or value-based backbone.

Part II: Coordination, competition, and communication

Behavioral structure

Once the algorithmic family is chosen, the central question becomes what the agents must coordinate, compete over, or communicate.

One-sentence idea

Multi-agent performance depends not only on the learning rule, but also on whether agents must align incentives, anticipate opponents, or exchange useful information.

A tiny coordination game: why exploration alone is not enough

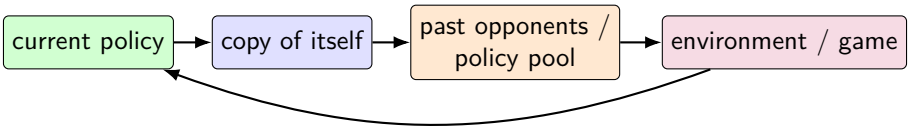
- Suppose two cooperative agents must independently choose whether to hunt stag or hare.
- The globally best outcome requires coordinated commitment, not just individual greed.
- MARL must discover policies that are jointly good, not merely locally safe.

	Stag	**Hare**
Stag	$(4, 4)$	$(0, 3)$
Hare	$(3, 0)$	$(2, 2)$

Lesson

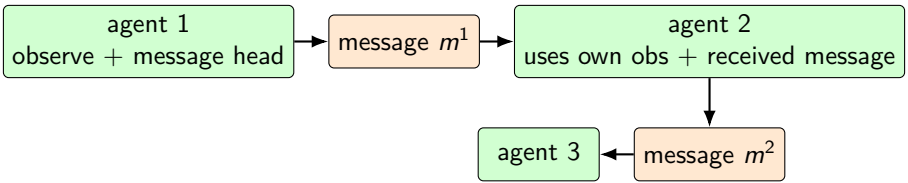
The high-payoff equilibrium $(\text{stag}, \text{stag})$ exists, but it is fragile during learning. If coordination breaks even slightly, agents may retreat to the safer but worse equilibrium.

Self-play: a practical recipe for competitive learning



- Train against copies of the current policy, past versions, or a league of opponents.
- This avoids overfitting to one frozen adversary and produces more robust strategies.
- In zero-sum settings, self-play is often the conceptual bridge from RL to approximate game solving.

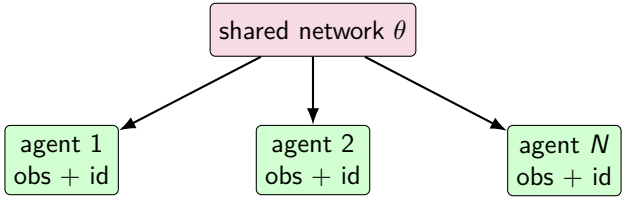
Learning to communicate



- Communication can be explicit (messages) or implicit (actions that reveal intent).
- It is most useful when observations are partial and coordination depends on hidden information.
- Learning communication raises extra questions: bandwidth, differentiability, noise, and whether the message protocol generalizes.

Parameter sharing and symmetry

- When agents are homogeneous, we often share the same network parameters across agents.
- Parameter sharing improves sample efficiency, reduces memory cost, and encourages permutation invariance.
- Agent ID, role embeddings, or local context can still differentiate behavior when the shared backbone is used.



Typical use

Parameter sharing is common in cooperative benchmarks with many similar agents, especially when combined with value factorization or MAPPO-style training.

A practical MARL training loop

- The main difference from single-agent RL is that data collection, target construction, and evaluation must account for all agents in the game.
- Parallel environment rollout is especially helpful because MARL data can be noisy and expensive.

Replay-based MARL vs on-policy MARL

Replay-based / off-policy

- reuses data aggressively
- often more sample-efficient
- needs stronger stabilization under non-stationarity
- typical examples: QMIX, MADDPG

On-policy / fresh-rollout

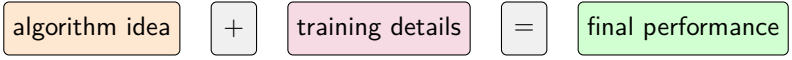
- learns from recent trajectories
- less stale data distribution
- often simpler to reason about
- typical example: MAPPO

Tradeoff

Off-policy methods save samples but must fight stale experience; on-policy methods throw away old data but often gain simplicity and stability.

Stabilization tricks that matter in practice

- parameter sharing for homogeneous agents
- target networks / Polyak averaging (off-policy methods)
- reward normalization and advantage normalization
- action masking when some actions are invalid
- curriculum learning or easier maps first
- opponent pools / self-play history instead of one fixed opponent
- careful rollout length and minibatch design

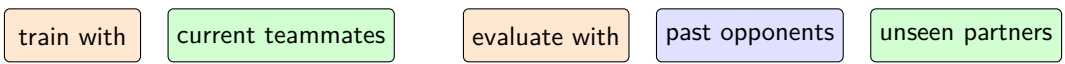


Practical mindset

Many MARL papers look different algorithmically, but a large fraction of performance differences also come from training protocol, implementation details, and benchmark-specific engineering.

Evaluation protocol: the minimum scientific standard

- Use multiple random seeds: MARL variance is often even higher than single-agent RL.
- Separate training partners/opponents from evaluation partners/opponents when possible.
- Report environment steps, number of agents, reward structure, observability assumptions, and evaluation frequency.
- For competitive settings, evaluate against a pool – not just the final self-play partner.



Do not overclaim

A policy that only works with the exact teammates or opponents it co-trained with may have learned brittle co-adaptation rather than generally useful behavior.

Metrics beyond mean return

Team return

Does the population solve the task at all?

Fairness / load balance

Does one agent carry the whole team?

Communication cost

If messages are used, what is the bandwidth / sparsity cost?

Win rate / success rate

Often easier to interpret than raw reward.

Robustness

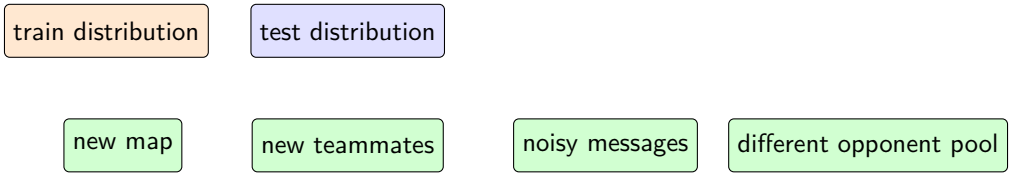
How sensitive is performance to partner or opponent changes?

Exploitability / regret

In competitive games, how vulnerable is the policy?

Generalization and robustness matter more than leaderboard numbers

- Ask whether the policy transfers to new maps, new teammate populations, or unseen opponent styles.
- Test ablations such as sensor noise, delayed communication, missing agents, or changed reward weights.
- A strong MARL system should remain coordinated under distribution shift, not only under the exact training population.



Interpretation

MARL can silently overfit to co-trained partners or to narrow game dynamics. Robust evaluation is part of the algorithm, not an afterthought.

Common MARL failure modes and debugging clues

- One agent dominates the reward while others become lazy.
- Training reward improves, but performance collapses with unseen teammates/opponents.
- Replay data becomes too stale because the rest of the population changed.
- Communication channel is ignored or overused.
- Agents converge to a safe but suboptimal equilibrium instead of coordinated high reward.

Debug checklist

Check reward decomposition, teammate/opponent diversity, observation design, invalid actions, action masks, and whether evaluation really uses the intended execution-time information only.

Sanity test

Always compare against a simple baseline: independent learners, parameter sharing only, or a hand-coded coordination heuristic if one exists.

Where MARL appears in practice

Traffic signal control

many agents, local observations, coupled rewards

Autonomous driving interaction

many agents, local observations, coupled rewards

Energy systems and markets

strategic interaction and decentralized decisions

Swarm / multi-robot coordination

many agents, local observations, coupled rewards

Wireless resource allocation

strategic interaction and decentralized decisions

Game-playing and simulation

strategic interaction and decentralized decisions

Course-level takeaway

If your domain has multiple decision-makers with coupled dynamics or rewards, ask whether it is really a single-agent control problem – or a multi-agent game in disguise.

Mini-exercise: model your domain as a stochastic game

Question 1

Who are the agents?

Question 2

What does each agent observe locally?

Question 3

What are the individual and/or team rewards?

Question 4

What actions are decentralized at execution time?

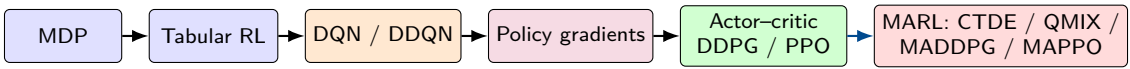
Question 5

Is the setting cooperative, competitive, or mixed?

Question 6

Would CTDE be realistic in training?

Conceptual map: from Module 1 to Module 5



- Module 5 does not replace the earlier material; it reuses all of it under a harder multi-agent game structure.
- Bellman ideas, value functions, actor–critic updates, and evaluation discipline still matter – but now they must survive interaction with other learners.

Summary: what to retain from this module

- A stochastic / Markov game generalizes an MDP to multiple learning agents.
- The central MARL difficulties are non-stationarity, credit assignment, partial observability, and combinatorial joint actions.
- CTDE is the dominant practical idea: use centralized information during training, but keep decentralized execution.
- Cooperative MARL often uses value factorization (VDN, QMIX) or centralized critics (COMA, MAPPO).
- Mixed and continuous-action settings often favor centralized-critic actor–critic methods such as MADDPG.
- Evaluation must test coordination robustness, not just average reward on the training population.

Suggested reading

- Littman, M. L. (1994). *Markov Games as a Framework for Multi-Agent Reinforcement Learning*.
- Buşoniu, Babuška, and De Schutter (2008). *A Comprehensive Survey of Multiagent Reinforcement Learning*.
- Sunehag et al. (2017/2018). *Value-Decomposition Networks for Cooperative Multi-Agent Learning*.
- Rashid et al. (2018). *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*.
- Foerster et al. (2018). *Counterfactual Multi-Agent Policy Gradients*.
- Lowe et al. (2017). *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*.
- Yu et al. (2022). *The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games*.

Suggested usage

Read the first two items for conceptual foundations, then use VDN/QMIX, COMA, MADDPG, and MAPPO papers as representative examples of the main MARL design families.